

```
GRANT ALL ON VEHICLE TO stud023;  
GRANT ALL ON CLIENT TO stud014;
```

```
CREATE SEQUENCE VEHICLE_ID_SEQ  
  START WITH 1000  
  INCREMENT BY 1;
```

```
CREATE TABLE VEHICLE (  
  V_ID NUMBER DEFAULT VEHICLE_ID_SEQ.NEXTVAL,  
  V_MODEL VARCHAR2(32),  
  V_MAKE VARCHAR2(32),  
  COST_PER_DAY NUMBER,  
  CONSTRAINT V_ID_PK PRIMARY KEY (V_ID)  
);
```

```
CREATE SEQUENCE CLIENT_ID_SEQUENCE  
  MINVALUE 1  
  START WITH 1  
  INCREMENT BY 1;
```

```
CREATE TABLE CLIENT (  
  C_ID NUMBER DEFAULT CLIENT_ID_SEQUENCE.NEXTVAL PRIMARY KEY,  
  F_NAME VARCHAR2(16),  
  L_NAME VARCHAR2(16),  
  DOB DATE,  
  CITY VARCHAR2(16),  
  GENDER CHAR(1)  
);
```

```
CREATE SEQUENCE RESERVATION_ID_SEQ  
  START WITH 100  
  INCREMENT BY 1;
```

```
CREATE TABLE RESERVATION (  
  R_ID NUMBER DEFAULT RESERVATION_ID_SEQ.NEXTVAL,  
  START_DATE DATE,  
  END_DATE DATE,  
  C_ID NUMBER,  
  V_ID NUMBER,  
  TOTAL_COST NUMBER,  
  CONSTRAINT R_ID_FK PRIMARY KEY (R_ID),  
  CONSTRAINT C_ID_FK FOREIGN KEY (C_ID) REFERENCES stud023.CLIENT(C_ID)  
);
```

```
CREATE SEQUENCE CLIENT_ID_SEQUENCE
  MINVALUE 1
  START WITH 1
  INCREMENT BY 1;
```

```
CREATE TABLE CLIENT (
  C_ID NUMBER DEFAULT CLIENT_ID_SEQUENCE.NEXTVAL,
  F_NAME VARCHAR2(16),
  L_NAME VARCHAR2(16),
  DOB DATE,
  CITY VARCHAR2(16),
  GENDER CHAR(1),
  CONSTRAINT PK_C_ID PRIMARY KEY(C_ID)
);
```

```
CREATE TABLE BLACK_LIST (
  C_ID NUMBER,
  F_NAME VARCHAR2(16),
  L_NAME VARCHAR2(16),
  DOB DATE,
  CITY VARCHAR2(16),
  GENDER CHAR(1),
  REASON VARCHAR2(128),
  ADDED_BY VARCHAR2(16),
  ADDED_DATE DATE,
  CONSTRAINT FK_C_ID_BL FOREIGN KEY(C_ID) REFERENCES CLIENT(C_ID)
);
```

```
CREATE TABLE STATS (
  YEAR NUMBER,
  C_ID NUMBER,
  AGE NUMBER,
  GENDER CHAR(1),
  TOT_RESERVATIONS INT,
  TOT_DAYS_OF_RESERVATIONS INT,
  TOT_COST_OF_RESERVATIONS INT,
  CONSTRAINT FK_C_ID_S FOREIGN KEY(C_ID) REFERENCES CLIENT(C_ID)
);
```

```
CREATE TABLE ARCHIVED_RESERVATION (
  R_ID NUMBER PRIMARY KEY,
  START_DATE DATE,
  END_DATE DATE,
  C_ID NUMBER ,
```

```

V_ID NUMBER,
TOTAL_COST NUMBER,
ADDED_BY VARCHAR2(128),
ADDED_DATE DATE DEFAULT SYSDATE,
FOREIGN KEY (C_ID) REFERENCES CLIENT(C_ID),
FOREIGN KEY (V_ID) REFERENCES VEHICLE(V_ID)
);

```

```

CREATE OR REPLACE FUNCTION isBlackListed(c_id CLIENT.C_ID%type) RETURN
BOOLEAN
IS
    clients NUMBER;
BEGIN
    SELECT COUNT(*) INTO clients FROM BLACK_LIST WHERE c_id = C_ID;
    RETURN clients > 0;
END;
/

```

```

CREATE OR REPLACE PROCEDURE BLACKLIST_CLIENT (
    p_C_ID IN NUMBER
)
AS
BEGIN
    INSERT INTO BLACK_LIST (C_ID, F_NAME, L_NAME, DOB, CITY, GENDER,
REASON, ADDED_BY, ADDED_DATE)
        SELECT CLIENT.C_ID, CLIENT.F_NAME, CLIENT.L_NAME, CLIENT.DOB,
CLIENT.CITY, CLIENT.GENDER,
        'Blacklisted', 'Admin', SYSDATE
        FROM CLIENT
        WHERE CLIENT.C_ID = p_C_ID;
END;
/

BEGIN
    BLACKLIST_CLIENT(1);
END;
/

```

```

CREATE OR REPLACE PROCEDURE GENERATE_STATS (
p_C_ID IN NUMBER)
AS
    V_TOT_RESERVATIONS INT := 0;
    V_TOT_DAYS_OF_RESERVATIONS INT := 0;
    V_TOT_COST_OF_RESERVATIONS INT := 0;
    V_GENDER CHAR(1);
BEGIN
    SELECT COUNT(R_ID) INTO V_TOT_RESERVATIONS FROM RESERVATION
WHERE C_ID = p_C_ID;
    DBMS_OUTPUT.PUT_LINE('Total Reservations: ' || V_TOT_RESERVATIONS);

    SELECT SUM(END_DATE - START_DATE) INTO V_TOT_DAYS_OF_RESERVATIONS
FROM RESERVATION WHERE C_ID = p_C_ID;
    DBMS_OUTPUT.PUT_LINE('Total Days Of Reservations: ' ||
V_TOT_DAYS_OF_RESERVATIONS);

    SELECT SUM(TOTAL_COST) INTO V_TOT_COST_OF_RESERVATIONS FROM
RESERVATION WHERE C_ID = p_C_ID;
    DBMS_OUTPUT.PUT_LINE('Total Cost of Reservations: ' ||
V_TOT_COST_OF_RESERVATIONS);

    SELECT GENDER INTO V_GENDER
FROM CLIENT
WHERE C_ID = p_C_ID;

    INSERT INTO STATS (YEAR, C_ID, GENDER, TOT_RESERVATIONS,
TOT_DAYS_OF_RESERVATIONS, TOT_COST_OF_RESERVATIONS)
VALUES (2024, p_C_ID, V_GENDER, V_TOT_RESERVATIONS,
V_TOT_DAYS_OF_RESERVATIONS, V_TOT_COST_OF_RESERVATIONS);

END;
/

```

```

CREATE OR REPLACE TRIGGER CHECK_AGE
BEFORE INSERT ON RESERVATION
FOR EACH ROW
DECLARE
    V_DOB DATE;
    V_AGE INT;
BEGIN
    SELECT DOB INTO V_DOB FROM CLIENT WHERE C_ID = :NEW.C_ID;
    V_AGE := FLOOR(MONTHS_BETWEEN(SYSDATE, V_DOB)/12);

```

```

        IF V_AGE < 18 THEN
            RAISE_APPLICATION_ERROR(-20003, 'Client must be at least 18 years old to make a
vehicle reservation.');
```

```
        END IF;
```

```
END;
```

```
/
```

```
CREATE OR REPLACE FUNCTION isBlackListed(c_id CLIENT.C_ID%type) RETURN
BOOLEAN
IS
```

```
    clients NUMBER;
```

```
BEGIN
```

```
    SELECT COUNT(*) INTO clients FROM BLACK_LIST WHERE c_id = C_ID;
```

```
    RETURN clients > 0;
```

```
END;
```

```
/
```

```
CREATE OR REPLACE TRIGGER ValidateReservation
```

```
BEFORE INSERT ON RESERVATION
```

```
FOR EACH ROW
```

```
DECLARE
```

```
    v_reason BLACK_LIST.REASON%TYPE;
```

```
BEGIN
```

```
    IF isBlackListed(:NEW.C_ID) THEN
```

```
        SELECT REASON INTO v_reason
```

```
        FROM BLACK_LIST
```

```
        WHERE C_ID = :NEW.C_ID;
```

```
        RAISE_APPLICATION_ERROR(-20001, 'Client is blacklisted: ' || v_reason);
```

```
    ELSIF isIssued(:NEW.V_ID, :NEW.START_DATE, :NEW.END_DATE) THEN
```

```
        RAISE_APPLICATION_ERROR(-20002, 'Vehicle ' || :NEW.V_ID || ' is already issued to
another client.');
```

```
    END IF;
```

```
END;
```

```
/
```

```
CREATE OR REPLACE FUNCTION isIssued(v_id IN VEHICLE.V_ID%TYPE, start_date IN
DATE, end_date IN DATE) RETURN BOOLEAN
```

```
IS
```

```
    conflicts NUMBER;
```

```
BEGIN
```

```

SELECT COUNT(*) INTO conflicts
FROM RESERVATION
WHERE V_ID = v_id
AND (
    (start_date BETWEEN START_DATE AND END_DATE) OR
    (end_date BETWEEN START_DATE AND END_DATE) OR
    (START_DATE BETWEEN start_date AND end_date) OR
    (END_DATE BETWEEN start_date AND end_date)
);
RETURN conflicts > 0;
END;
/

CREATE OR REPLACE PROCEDURE FIND_ACTIVE_CLIENTS (
    p_START_DATE IN DATE,
    p_END_DATE IN DATE
) AS
    CURSOR active_clients_cursor IS
        SELECT DISTINCT C.C_ID, C.F_NAME, C.L_NAME, R.START_DATE, R.END_DATE
        FROM CLIENT C
        JOIN RESERVATION R ON C.C_ID = R.C_ID
        WHERE R.START_DATE <= p_END_DATE AND R.END_DATE >= p_START_DATE;

    v_client_id CLIENT.C_ID%TYPE;
    v_first_name CLIENT.F_NAME%TYPE;
    v_last_name CLIENT.L_NAME%TYPE;
    v_start_date DATE;
    v_end_date DATE;
BEGIN
    DBMS_OUTPUT.PUT_LINE('Active Clients for the specified period:');
    DBMS_OUTPUT.PUT_LINE('-----');
    FOR client_record IN active_clients_cursor LOOP
        DBMS_OUTPUT.PUT_LINE('Client ID: ' || client_record.C_ID ||
            ', Name: ' || client_record.F_NAME || ' ' || client_record.L_NAME ||
            ', Reservation Period: ' || client_record.START_DATE || ' to ' ||
client_record.END_DATE);
    END LOOP;
END;
/

CREATE OR REPLACE PROCEDURE YearlyArchiveReservations AS
BEGIN
    INSERT INTO ARCHIVED_RESERVATION (R_ID, START_DATE, END_DATE, C_ID, V_ID,
TOTAL_COST, ADDED_BY)

```

```
SELECT R_ID, START_DATE, END_DATE, C_ID, V_ID, TOTAL_COST, 'system'
FROM RESERVATION
WHERE END_DATE IS NOT NULL
AND END_DATE < ADD_MONTHS(SYSDATE, -12);
```

```
DELETE FROM RESERVATION
WHERE END_DATE IS NOT NULL
AND END_DATE < ADD_MONTHS(SYSDATE, -12);
```

```
COMMIT;
```

```
EXCEPTION
  WHEN OTHERS THEN
    ROLLBACK;
    RAISE;
END;
/
```

```
CREATE OR REPLACE FUNCTION calculateTotalRevenue(v_id VEHICLE.V_ID%type)
RETURN NUMBER
IS
  v_sum_res NUMBER;
  v_sum_arch NUMBER;
BEGIN
  SELECT COUNT(*) INTO v_sum_res FROM RESERVATION WHERE v_id = V_ID;
  SELECT COUNT(*) INTO v_sum_arch FROM ARCHIVED_RESERVATION WHERE v_id =
V_ID;
  RETURN v_sum_res + v_sum_arch;
END;
/
```

```
CREATE OR REPLACE FUNCTION CreateReservation(
  client_id IN NUMBER,
  vehicle_id IN NUMBER,
  start_date IN DATE,
  end_date IN DATE
) RETURN NUMBER
IS
  reservation_id NUMBER;
  estimated_budget NUMBER;
  duration_days NUMBER;
BEGIN
  duration_days := end_date - start_date + 1;
  SELECT COST_PER_DAY * duration_days INTO estimated_budget
```

```

FROM VEHICLE
WHERE V_ID = vehicle_id;

INSERT INTO RESERVATION (START_DATE, END_DATE, C_ID, V_ID, TOTAL_COST)
VALUES (start_date, NULL, client_id, vehicle_id, estimated_budget)
RETURNING R_ID INTO reservation_id;

DBMS_OUTPUT.PUT_LINE('Estimated budget: $' || estimated_budget);
RETURN reservation_id;
EXCEPTION
  WHEN NO_DATA_FOUND THEN
    DBMS_OUTPUT.PUT_LINE('Error: Vehicle not found. ');
    RETURN NULL;
  WHEN DUP_VAL_ON_INDEX THEN
    DBMS_OUTPUT.PUT_LINE('Error: Duplicate reservation ID. ');
    RETURN NULL;
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('An unexpected error occurred: ' || SQLERRM);
    RETURN NULL;
END;
/

CREATE OR REPLACE FUNCTION calculateTotal (
  start_date DATE,
  end_date DATE,
  v_rate VEHICLE.COST_PER_DAY%TYPE
) RETURN NUMBER
IS
  christmas DATE := TO_DATE('25-DEC', 'DD-MON');
  rate NUMBER := 1;
  total_days NUMBER;
BEGIN
  total_days := end_date - start_date + 1;
  IF total_days > 10 THEN
    rate := rate - 0.10;
  END IF;

  IF christmas BETWEEN start_date AND end_date THEN
    rate := rate - 0.20;
  END IF;
  RETURN rate*v_rate*total_days;
END;
/

```



```

CREATE OR REPLACE PROCEDURE returnCar(
    reservation_id IN RESERVATION.R_ID%TYPE,
    client_id IN CLIENT.C_ID%TYPE,
    is_paid IN NUMBER,
    car_damaged IN NUMBER
) AS
    start_date DATE;
    end_date DATE;
    client_id_res CLIENT.C_ID%TYPE;
    daily_rate VEHICLE.COST_PER_DAY%TYPE;
    total_cost NUMBER;
BEGIN
    IF reservation_id IS NOT NULL THEN
        SELECT START_DATE, END_DATE, C_ID, TOTAL_COST INTO start_date, end_date,
        client_id_res, daily_rate
        FROM RESERVATION
        WHERE R_ID = reservation_id;
    ELSE
        SELECT START_DATE, END_DATE, C_ID, TOTAL_COST INTO start_date, end_date,
        client_id_res, daily_rate
        FROM RESERVATION
        WHERE C_ID = client_id
        ORDER BY END_DATE DESC
        FETCH FIRST 1 ROWS ONLY;
    END IF;

    total_cost := calculateTotal(start_date, end_date, daily_rate);

    IF is_paid = 0 OR car_damaged = 1 THEN
        INSERT INTO BLACK_LIST (C_ID, F_NAME, L_NAME, DOB, CITY, GENDER, REASON,
        ADDED_BY, ADDED_DATE)
        SELECT C_ID, F_NAME, L_NAME, DOB, CITY, GENDER,
        CASE
            WHEN is_paid = 0 THEN 'Non-payment'
            WHEN car_damaged = 1 THEN 'Damaged vehicle'
        END,
        'system', SYSDATE
        FROM CLIENT
        WHERE C_ID = client_id_res;

        DBMS_OUTPUT.PUT_LINE('Client has been added to the blacklist for: ' ||
        CASE
            WHEN is_paid = 0 THEN 'Non-payment'
            WHEN car_damaged = 1 THEN 'Damaged vehicle'

```

```
                END);  
ELSE  
    DBMS_OUTPUT.PUT_LINE('Total cost after discount: $' || total_cost);  
END IF;  
END;  
/
```